The Bug Life-cycle States

0:05/6:52

Introduction

Imagine you find a bug in an app. You report it. Then... what happens? Where does it go? Who owns it now? And how does everyone on the team know what stage it is in?

Without a clear answer to those questions, bugs get lost, duplicated, or quietly forgotten. A developer might start working on a ticket that was already closed. A manager might assume a bug is fixed when it is still wide open. That kind of confusion slows teams down and erodes trust in the testing process.

That is exactly what the **bug life cycle** is designed to prevent. It is the shared map that every tester, developer, and project manager follows — from the moment a defect is discovered to the moment it is officially closed. When everyone follows the same life cycle, ownership is always clear, and nothing falls through the cracks.

By the end of this reading, you will be able to:

- **Identify** the four core stages of the bug life cycle: New, Active, Resolved, and Closed.
- **Distinguish** between bug status and bug resolution type.
- **Recognize** the seven resolution types — Duplicate, Won't Fix, Cannot Reproduce, Fixed, Known Issue, Not a Bug, and Deferred — and explain when each one applies.
- **Explain** how the life cycle keeps the whole team aligned on who owns a bug at any given moment.

The Core Workflow: Four Stages Every Bug Travels

Think of the bug life cycle as a relay race. Each stage hands ownership of the bug to a different person on the team. No one is left guessing what to do next — and no one can claim they did not know the bug was theirs to act on.



A horizontal flowchart depicts the four chronological stages of a bug's life cycle, progressing from left to right: it begins with the purple "New" stage where a tester reports the issue, moves to the pink "Active" stage where code brackets signify developer ownership, proceeds to the purple "Resolved" stage where a wrench and screwdriver icon indicates a fix or decision has been made, and concludes with the blue "Closed" stage showing a checkmark to denote that the bug is verified and done.

New

When you log a bug, it enters the **New** state. At this point, the bug has been reported but not yet reviewed by anyone else. Think of it as a ticket sitting in an inbox — it exists, but no one has picked it up yet. The tester owns this stage. Your responsibility here is to write a clear, complete report so that whoever reviews it next can make a quick, informed decision.

Active

Once a team lead or developer reviews the bug and confirms it is valid, it moves to **Active** (sometimes called "In Progress" or "Open"). This means the bug is acknowledged and someone is actively working on it. The developer now owns this stage. If the reviewer determines the ticket needs more information before work can begin, it may be sent back to the tester — which is why a thorough initial report saves everyone time.

Resolved

When the developer finishes their work — whether that means fixing the bug, deciding not to fix it, or making another determination — the bug moves to **Resolved**. This is where the **resolution type** is set (more on that below). At this point, ownership shifts back to the tester. It is now your job to verify whether the resolution is correct.

Closed

After the tester confirms the resolution — either the fix works, or the decision makes sense — the bug is marked **Closed**. This is the finish line. The life cycle is complete, and the ticket becomes part of the permanent record.

Brace yourself — the next section introduces seven resolution types, which might feel like a lot at first. But each one maps to a real situation you will encounter on the job, and they are easier to remember once you see them in context.

Resolution Types: How a Bug Gets "Resolved"

Not every bug ends with a fix. When a bug reaches the Resolved stage, the team assigns a **resolution type** — a label that explains exactly how and why the bug was closed. This is a separate decision from the status. The status tells you where the bug is in the workflow. The resolution type tells you what happened there.

Resolution Type	What It Means	Example
Fixed	The bug was reproduced and corrected by a developer.	A broken login button now works after a code change.
Duplicate	The same bug was already reported in another ticket.	Two testers logged the same checkout error separately.

Cannot Reproduce	The team could not make the bug happen again.	The crash only occurred once and never repeated.
Won't Fix	The team decided not to fix it — usually due to low impact or high cost.	A minor alignment issue in a rarely visited settings page.
Not a Bug	The reported behavior is actually working as intended.	A tester reported a feature as broken, but it was by design.
Known Issue	The bug is real and acknowledged, but a fix is not yet scheduled.	A performance lag the team is aware of but has deprioritized.
Deferred	The fix is valid but postponed to a future release.	A non-critical UI issue pushed to the next sprint.

Each resolution type tells a story. **Fixed** means the problem is solved. **Won't Fix** means the team made a deliberate call. **Deferred** means it is on the radar but not yet urgent. **Cannot Reproduce** means more evidence may be needed before the team can act.

As a tester, you play a role in several of these outcomes. If a bug comes back as "Cannot Reproduce," that is a signal to revisit your original report — were the steps clear enough? Was the environment documented? Strong reporting reduces the number of tickets that stall at this resolution. Together, these labels give everyone — testers, developers, and managers — a clear, shared record of every decision made.

Why the Life Cycle Matters

Without a shared life cycle, bugs get lost. The life cycle prevents that by making ownership and status visible to everyone at all times. It also creates an audit trail — a record of every state change, every decision, and every person involved. That trail is invaluable when a bug reappears after being closed, or when a release decision needs to be justified.

As a tester, your role is not just to find bugs — it is to move them through the system accurately. That means logging them clearly, verifying fixes thoroughly, and using the right resolution type when closing them out. Every step you take in the life cycle is a contribution to the team's shared understanding of the product's quality.

Conclusion

The bug life cycle is the backbone of any organized testing process. By understanding the four stages — **New, Active, Resolved, and Closed** — and the seven resolution types that explain how a bug was handled, you can track any defect from discovery to done. That clarity keeps your team aligned, your backlog honest, and your testing practice professional.